

Bug Hunt v2-2: Graph + Traversal

What the program does

This problem consists of **two modules** plus a driver:

- `graph.erl` — an undirected-graph ADT. A graph is represented as a list of `{Node, [Neighbor]}` pairs. Exports: `new/0`, `add_node/2`, `add_edge/3`, `nodes/1`, `neighbors/2`, `has_node/2`, `has_edge/3`, `node_count/1`, `edge_count/1`.
- `traversal.erl` — graph algorithms that operate on the ADT. Exports: `bfs/2`, `dfs/2`, `shortest_path/3`, `connected_components/1`, `reachable_from/2`.
- `main.erl` — builds a fixed test graph (9 nodes, 7 undirected edges, 3 connected components, including an isolated node) and exercises every operation.

The test graph:

```
a ---- b
|      |
c ---- d ---- e
      |
      f

g ---- h

i      (isolated)
```

Intended graph behavior:

- The graph is *undirected*: if `add_edge(G, A, B)` is called, then both B is in `neighbors(G, A)` and A is in `neighbors(G, B)`.
- `edge_count(G)` returns the number of undirected edges.
- `has_edge(G, A, B)` is symmetric: `has_edge(G, A, B)` equals `has_edge(G, B, A)`.

Intended traversal behavior:

- `bfs(G, Start)` and `dfs(G, Start)` return the visited nodes starting with `Start` in BFS or DFS visit order respectively.
- `shortest_path(G, From, To)` returns a list `[From, ..., To]` of minimum length, or the atom `no_path` if no such path exists.
- `connected_components(G)` returns a list of components, where every node in the graph appears in **exactly one** component. A node with no edges forms its own component (a singleton).
- `reachable_from(G, Start)` returns the set of nodes reachable from `Start`, including `Start` itself.

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

The catch

This program contains **exactly 4 bugs** that prevent it from producing the expected output. The bugs are all *semantic* — the program compiles without errors and runs to completion without crashing.

This problem is harder than the round-1 bug hunts in four specific ways:

1. **At least one bug's cause lives in a different file from where its symptom appears.** Most of the traversal output is wrong, but the traversal code is correct.
2. **Two of the bugs interact via masking.** Each one individually would produce visibly wrong output for one specific operation, but together they cancel for that operation. Fixing exactly one of them in isolation makes that operation visibly wrong — possibly in the *opposite* direction from what you would expect.
3. **One bug is hidden behind another.** In the as-shipped state, one bug's effect is buried inside another bug's much louder symptoms. The hidden bug only becomes crisply visible after you fix the louder one.
4. **One bug's tell is not a disagreement with the expected output, but a suspicious agreement between two outputs that are supposed to disagree.** Two algorithmically distinct operations produce identical results — and they shouldn't.

You may not run the program until you have submitted your answer. Trace by reading.

Expected output

```
Nodes: [a,b,c,d,e,f,g,h,i]
Node count: 9
Edge count: 7

Neighbors of a: [b,c]
Neighbors of d: [b,c,e,f]
Neighbors of i: []

has_edge(a, b): true
has_edge(b, a): true
has_edge(a, e): false
has_edge(d, g): false

BFS from a: [a,b,c,d,e,f]
BFS from f: [f,d,b,c,e,a]
BFS from i: [i]
DFS from a: [a,b,d,c,e,f]

Shortest path a -> e: [a,b,d,e]
Shortest path a -> f: [a,b,d,f]
Shortest path a -> g: no_path
Shortest path i -> a: no_path

Connected components: [[a,b,c,d,e,f],[g,h],[i]]
```

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

```
Component count: 3

Reachable from a: [a,b,c,d,e,f]
Reachable from g: [g,h]
Reachable from i: [i]
```

Notes on the expected output:

- **Neighbors** of **X** and **Nodes** are sorted in the driver before printing — do not treat the order as a bug.
- **Connected components** and the inner lists are sorted in the driver before printing.
- BFS and DFS visit order depend on the order neighbors were inserted. The driver adds edges in the order [(a,b), (a,c), (b,d), (c,d), (d,e), (d,f), (g,h)]. With `add_edge(G, A, B)` appending B to the end of A's neighbor list, the resulting neighbor lists are in insertion order: e.g., `neighbors(G, a) = [b, c]`, `neighbors(G, d) = [b, c, e, f]`. BFS and DFS process neighbors in that order. Use this to predict the visit order yourself.
- **Shortest path** `a -> e` and `a -> f` happen to go through `b` rather than `c` because `b` is enqueued before `c` (it comes first in `a`'s neighbor list). Either path would be a valid shortest path in general, but the driver expects the one shown.

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom. If a bug has no *direct* visible symptom (because another bug masks it), explain how you knew it was there anyway.